

Service Mesh Fundamentals: Providing Security, Reliability, and Observability to Kubernetes Applications

Author: Flyn

- Examples of existing solutions:
 - Istio
 - Linkerd
 - Consul
 - Kuma

Definition:

An infrastructure layer providing security, reliability, and observability at a platform level, uniformly across an entire application

The Microservice Arch

- Multiple binaries
- Multiple processes
- Multiple machines (usually)

Meshes **typically** work by adding a proxy next to each application pod

Proxies **mediate** and **measure** all communication

Proxies mediate communication, allowing them to enforce rules

- mTLS
- advanced load balancing
- retries, timeouts, etc.

Proxies measure communication

- Discover and display the call graph
- Measure and publish golden metrics (latency, success rate, traffic)

(In)security

- Eavesdropping:
 - An evildoer can snoop on cleartext communications
 - Encryption
- Tampering:
 - An evildoer can change what's being sent
 - Integrity checking
- Identity theft:

- An evildoer can pretend to be a legitimate workload
- Authentication

Mesh security

Meshes provide all three (Encryption, Integrity checking, Authentication)

Most commonly, the proxies enforce mTLS for all communication.

Some meshes do use lower-level mechanism like WireGuard or IPsec.

Mesh can't help if application has its own vulnerabilities

(Un)reliability

- The network can fail
- Services can fail
- Things can get overloaded

Reliability

- Retries
- Timeouts
- Circuit breakers

The mesh can't directly improve reliability of things inside a single application Pod

Mesh observability

- The mesh is in perfect place to measure everything
- In particular, real-time telemetry can reveal the call graph of the application
- Easily collects golden metrics:
 - Success rates
 - Latency
 - Traffic volume

Service meshes control communications

- All meshes work by taking over network communications between workloads in the application
- Not all meshes do this in exactly the same way

Service meshes focus on workloads

- Meshes operate at the level of the workloads, not at the level of end users
- A mesh can guarantee that a given workload is what it claims to be; the application must worry about the user

Commonalities

- All modern meshes must:
 - Manage communication to add security, reliability, and observability to an entire application at the platform level
 - Provide a way to tackle ingress

- Allow multicluster operation -Be able to work with non-K8s workloads
- All meshes must manage communication to provide:
 - Secure communication between application pods (mTLS)
 - Fine-grained workload authentication and authorization
 - Per-request load balancing
 - Retries, circuit breakers, timeouts, etc.
 - Uniform metrics across the entire application

Managing Ingress

Ingress: bringing traffic in from outside the cluster

- The ingress problem is fundamental for cloud native world
- Popular mesh solutions work with ingress controller to solve this problem

Multicluster

In case of multicluster we rely on the mesh to handle communication between clusters in the same way that we handle communication within a cluster. Typically, you'll end up with an instance of the control plane running on each of your clusters.

In 2023 Istio released Istio Ambient

- L4 (Transport) is handled in the per-Node Rust ztunnel proxy
- L7 (Application) is handled by an Envoy (the "waypoint")
- The point of Ambient is to try reduce resource usage and increase performance
- Probably you will use only one Envoy(connection between node) because ztunnels (between micro-services) are much lighter weight
- You do possibly run into some more operational complexity with this

Mastering Service Mesh

Authors: Anjali Khatri, Vikram Khatri

Service Mesh definition by William Morgan in 2017

A service mesh is a dedicated infrastructure layer for handling service-to-service communication. It's responsible for the reliable delivery of requests through the complex topology of services that comprise a modern, cloud-native application. In practice, the service mesh's implementation is an array of lightweight network proxies deployed alongside microservices, without the applications needing to be aware.

The service mesh could secure communication between the microservices with TLS. The benefit here is that each developer no longer has to implement TLS encryption and decryption that's specific to the language they are writing in.

The service mesh concept is a significant shift from earlier versions of DevOps, where operations were limited to software release management.

Smart endpoints: Service-to-service communication is done through the intelligent endpoints, which is a DNS record that resolves to a microservice. The use of DNS records facilitates one service to communicate with others, and this eliminates the load balancer between microservices.

Dumb pipes: Service-to-service communication uses basic network traffic protocols such as HTTP, REST, gRPC, and so on. This type of connection is opposed to a centralized smart pipe using the ESB/MQ of monolithic applications.

Martin Fowler, in his 2014 blog article

Service Mesh def. by Christian Posta

Service mesh is a decentralized application networking infrastructure between your services. This decoupling provides resiliency, security, observability, and routing control.

In a cloud-native environment, the service mesh has moved from Dev to Ops, and this is a significant shift because it lets the developers focus on their specialty

Site Reliability Engineering (SRE)

In the context of Kubernetes:

- Basic service mesh:
 - K8s provides basic service mesh "out of the box"
 - provides a round-robin balancing for requests to the target pods
 - dynamic function that manages iptables under the cover on each host (this process is transparent)
 - Pod is ready - the endpoints of the service are enabled to provide a connection from outside to the IP addresses of the pods.
- Advanced service mesh:
 - Istio, Linkerd, Consul, and so on can harness some advanced capabilities such as retry logic, timeouts, fallback, circuit breaking, and distributed tracing from it.

Emerging trends

- We can extend the same service mesh control plane to multiple Kubernetes clusters, provided that each cluster has its own distinct IP address range.
- We can bring VMs, bare-metal, or other monolithic applications into the service mesh for traffic management and telemetry.
- We can have multiple Kubernetes clusters, with each having their control plane replicating the state of each group.
- We can have a federated service mesh, where each cluster runs its own control and data plane.

Service Mesh rules:

- Observability: The control plane provides the observability of services running in the data plane.
- Routing: The routing rules for traffic management can be defined either graphically or through the use of configuration files and then pushed down from the control plane to all the data planes.
- Automatic scaling: The control plane services automatically scale to handle the increased workload.

- Separation of duties: The control plane UI allows operations to manage the service mesh independent of the development team.
- Trust: It pushes down secure communication protocols to the data planes and provides automatic renewal and management of certificates.
- Automatic service registration and discovery: The control plane integrates with the Kubernetes API server and discovers the service automatically as it's registered through application deployment procedures.
- Resilient: Pushes resiliency rules to all of the data planes. This acts as a sidecar proxy for traffic management.